

KLEENE'S THEOREM

Recap

We discussed 3 ways of defining a language.

- ① By Regular Expression
- ② Acceptance by finite Automata.
- ③ Acceptance by Transition Graphs.

Kleene's Theorem.

(very important in FA)

→ states;

If a language can be defined by any of the 3 ways as listed above, then, one way there can be (for every) defined the other two ways as well.

i.e All the 3 ways are equivalent.

Let us Proof this theorem.

Consisting of three parts.

- ① Every language defined by FA could be defined by TG.
- ② Every language defined by TG can be defined by a R.E.

③ Every language that can be defined by R.E can also be defined by FA.

Lets Begin

Proof of Part 1

This is the easiest.

Every finite Automata is itself already a transition graph.

∴ language defined by a FA has already been defined by a TG.

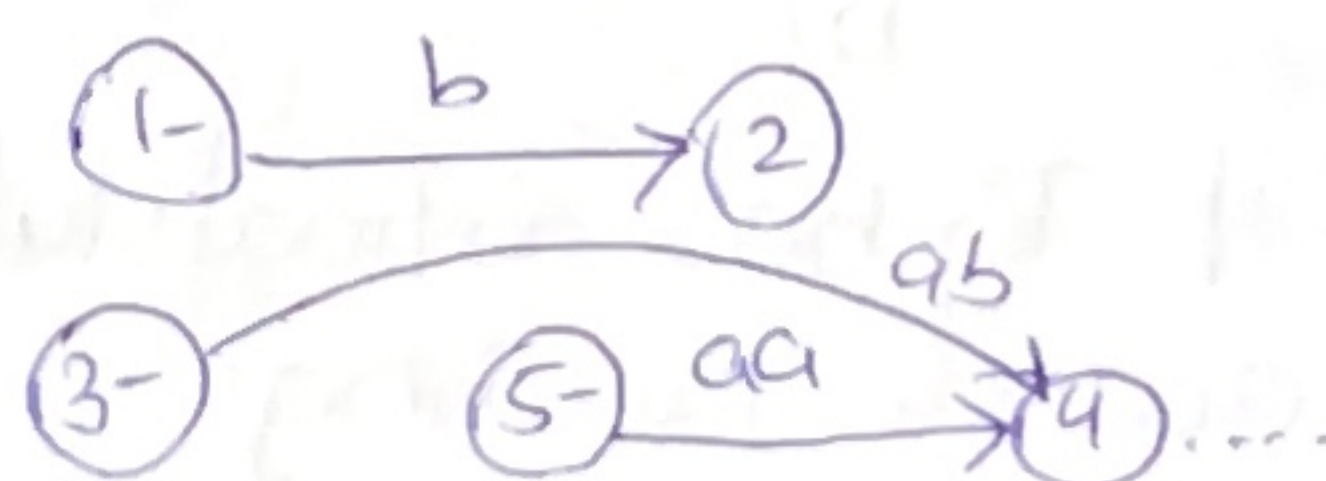
PROOF PART 2

Includes constructive algorithm

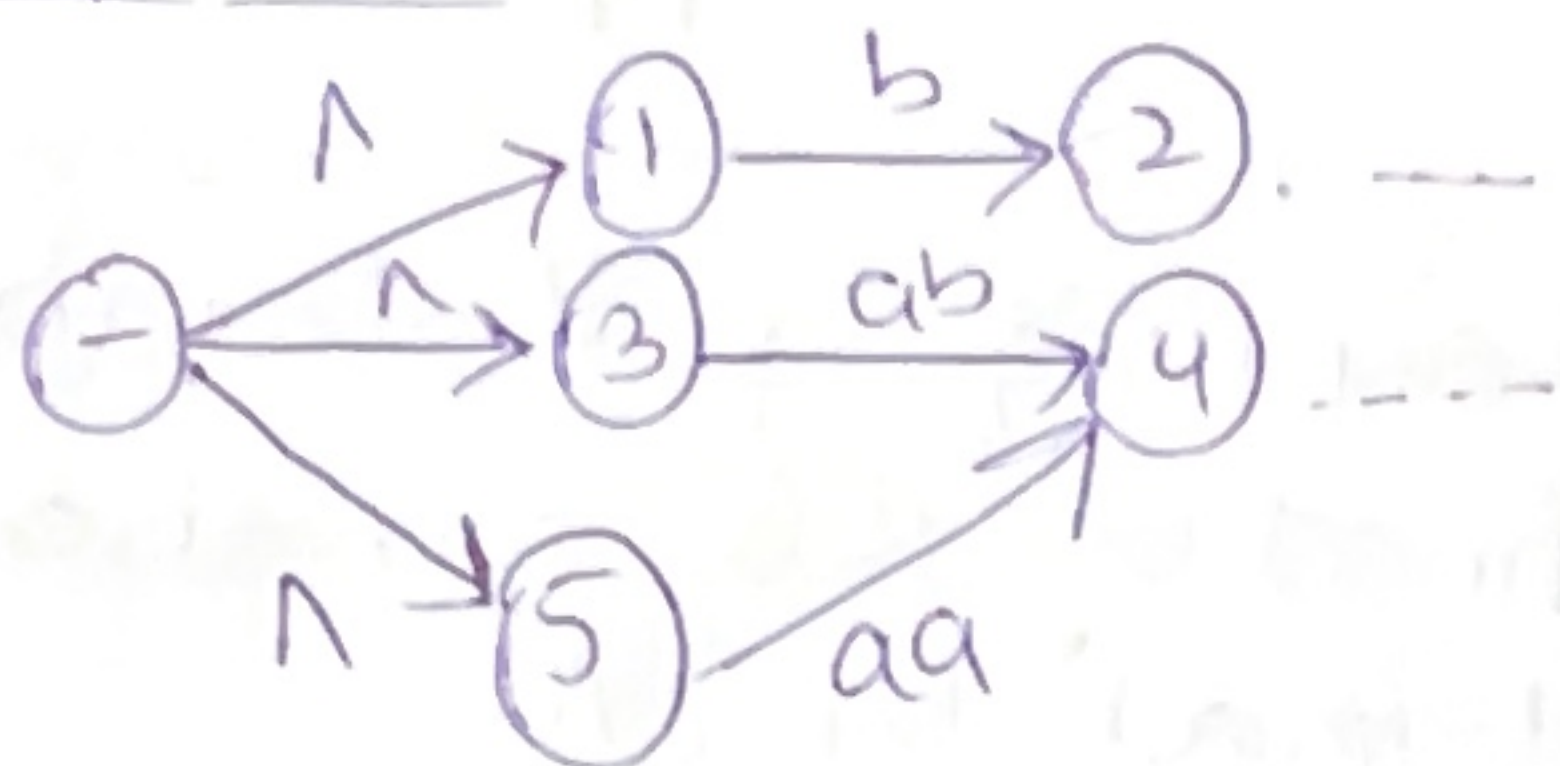
- it must satisfy the following criteria:

- ▷ must work for every TG
- ▷ must guarantee to finish the job in finite time.

* first lets simplify TG by keeping only one start state i.e :-



Step no.1.

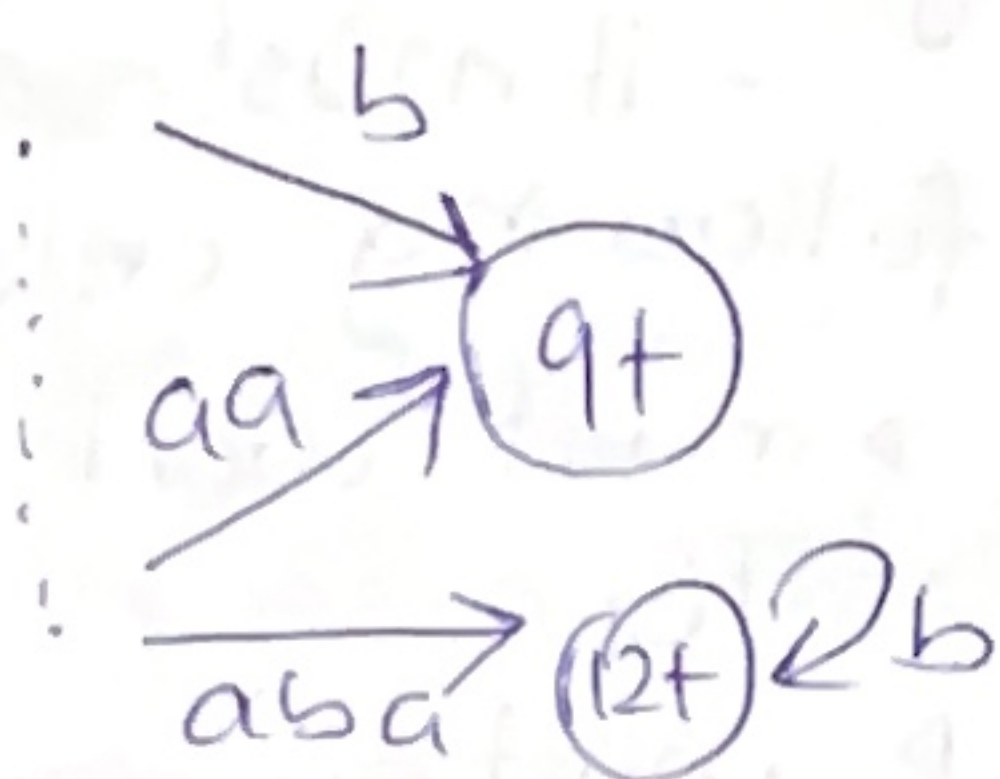


here we; Introduced a new state with minus sign and connected it with all the initial states and labelled the edge as Λ .

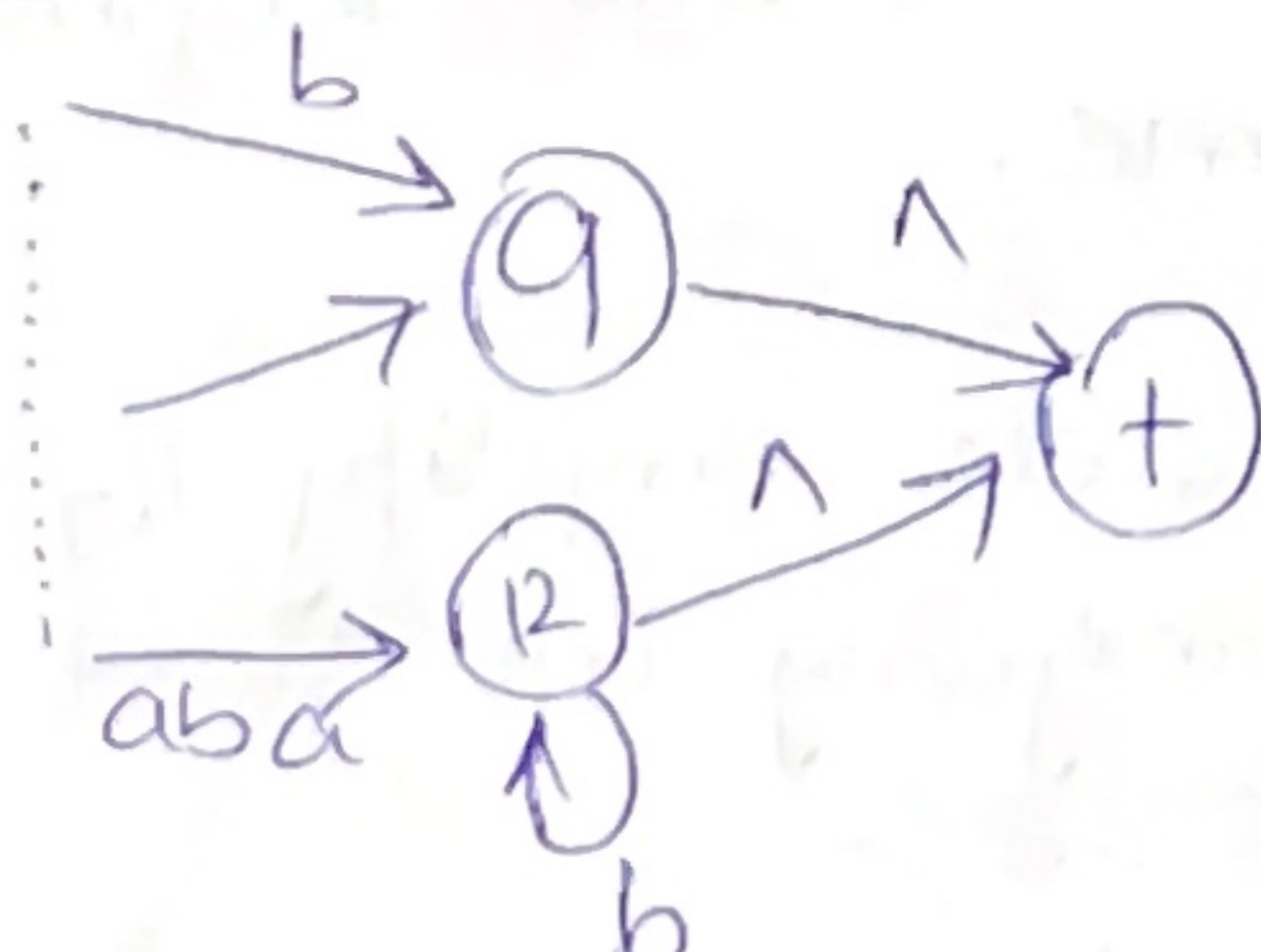
Finally we removed the minus sign from all the other states

step no.2 (second simplification).

TG can have many final states. Let us modify to have unique un-existable final state as;



becomes



Note if TG has no final states then it accepts no string

pg no.2.

and has no language so we should not produce any R.E other than the NULL expression ϕ

Q. What if TG have a state like $\oplus$?

we remove both signs from newly created states.



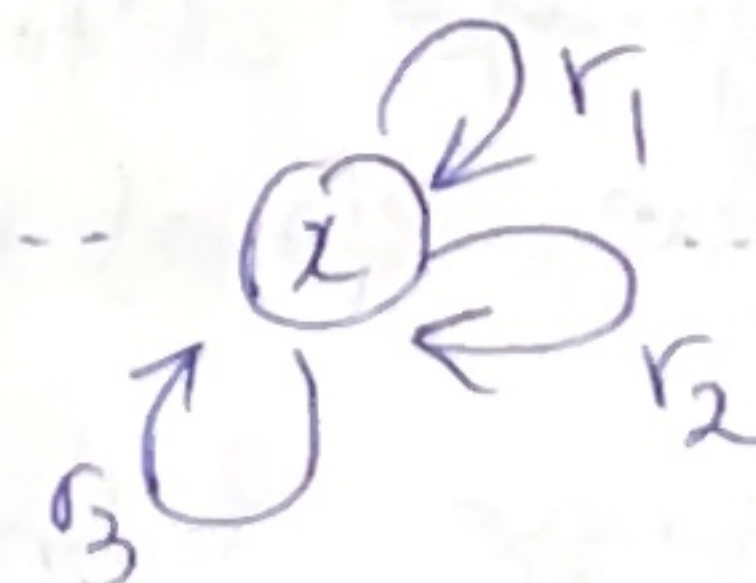
Note: Adding new states should not change the language of the TG



Step no.3 Build R.E.

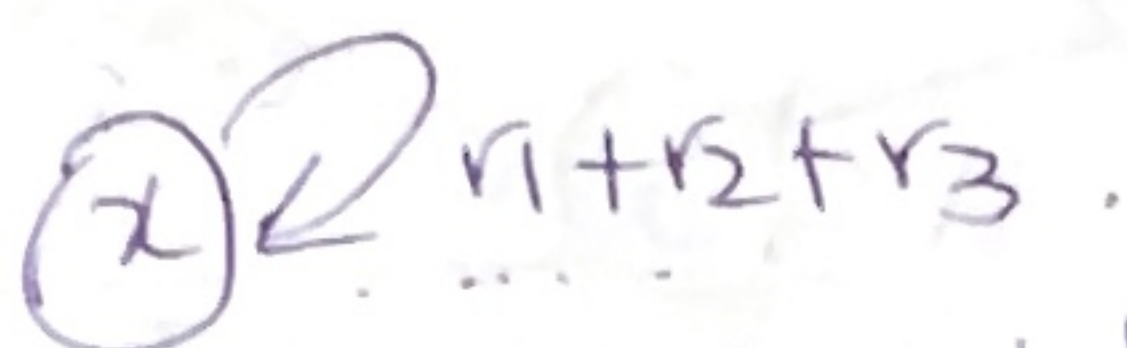
- Build R.E piece by piece.
- For this change TG into GTG.

i) Dealing with loops circling back into itself.



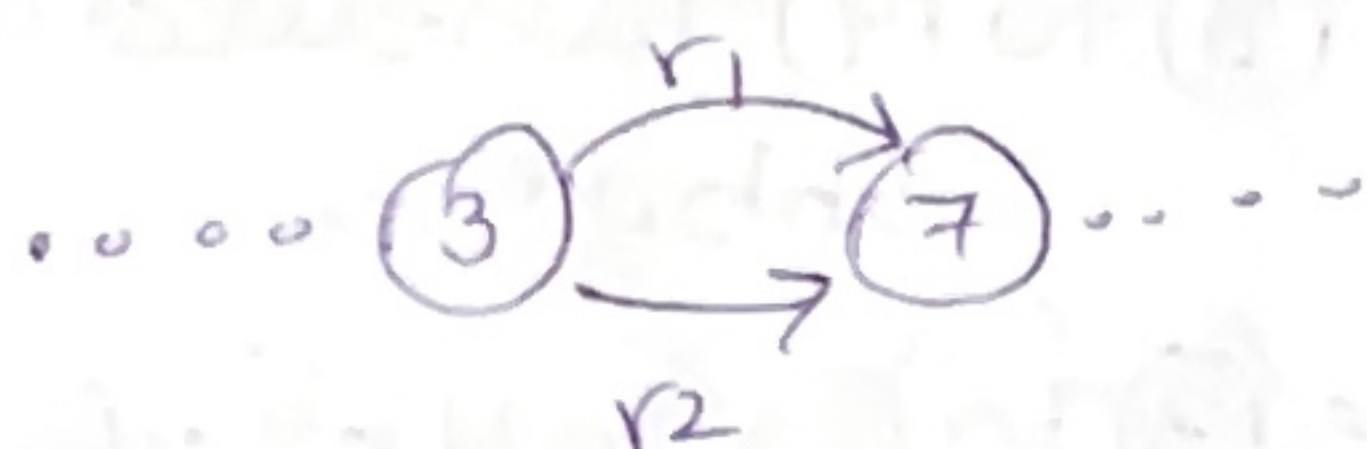
where r_1, r_2, r_3 are regular expressions or simple strings.

- replace these loops by one
- loop labelled with the a regular expression as.

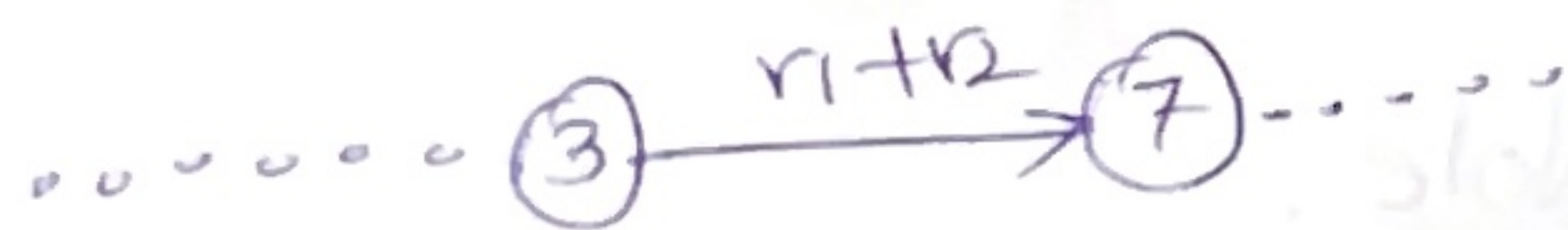


- * Do this for all similar states in your TG.

(ii) Dealing with 2 states that are connected by more than one edge going in the same direction.

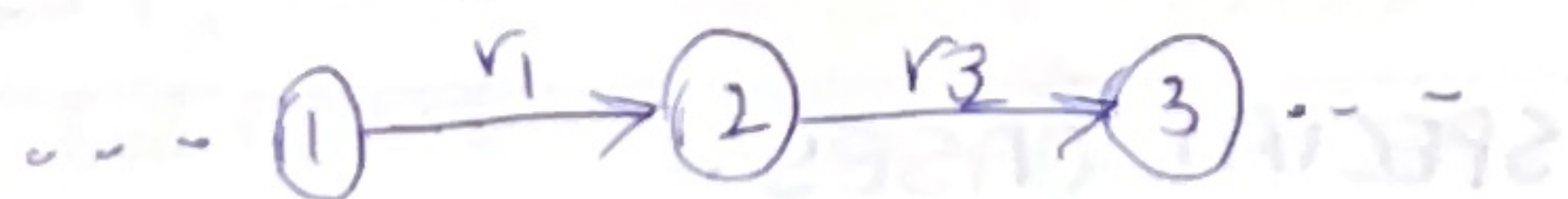


Changes into.



(iii) Bypass & state elimination.

If three states in a row connected by edges labelled as $r_1, r_2 \dots$ / RE or simple strings as

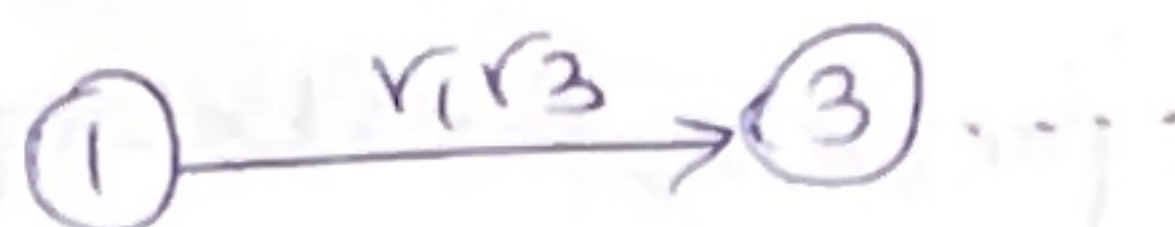


— Eliminate the middle state and go directly

from $(1) \rightarrow (3)$ and the new edge will be labelled

with a RE which is the concatenation of the previous labels

i.e

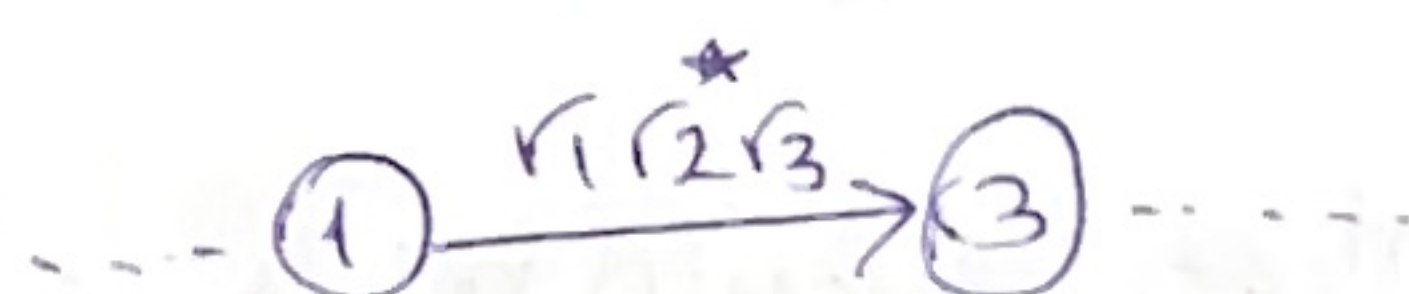


Note : This can only be done if the middle state has no loop going back to itself.

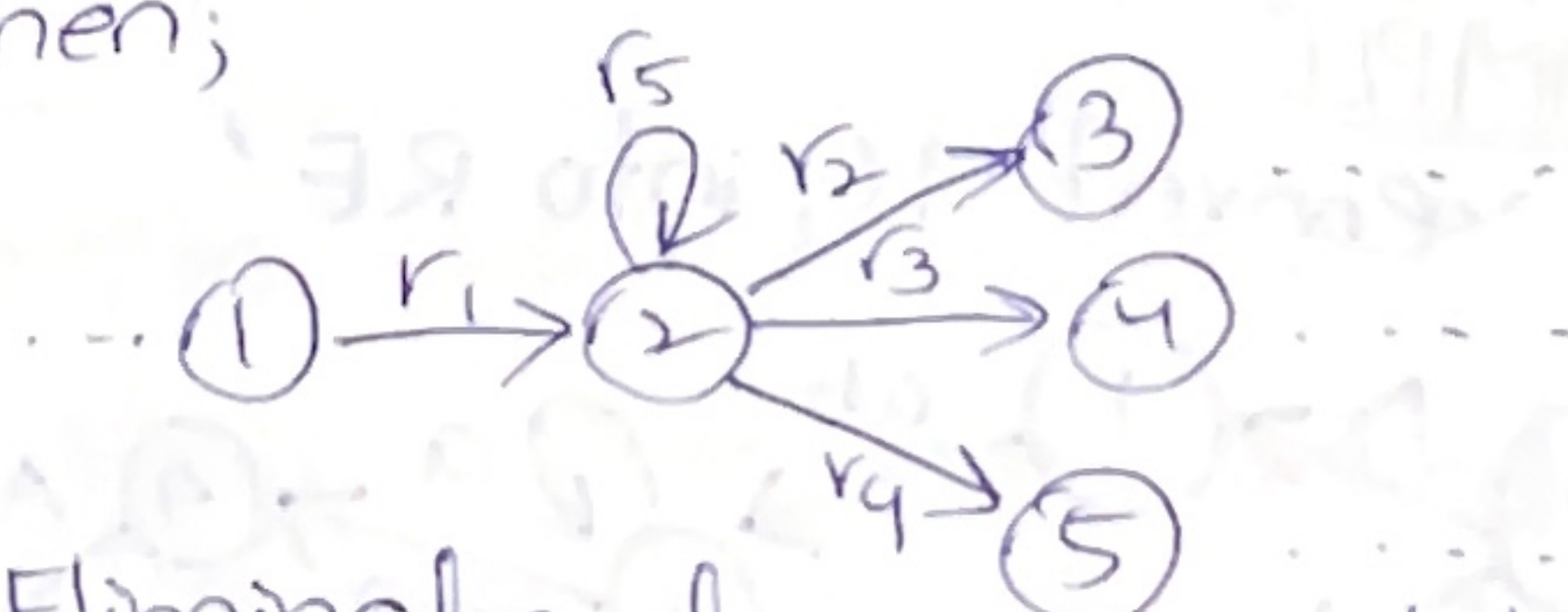
* if state (2) had a loop like,



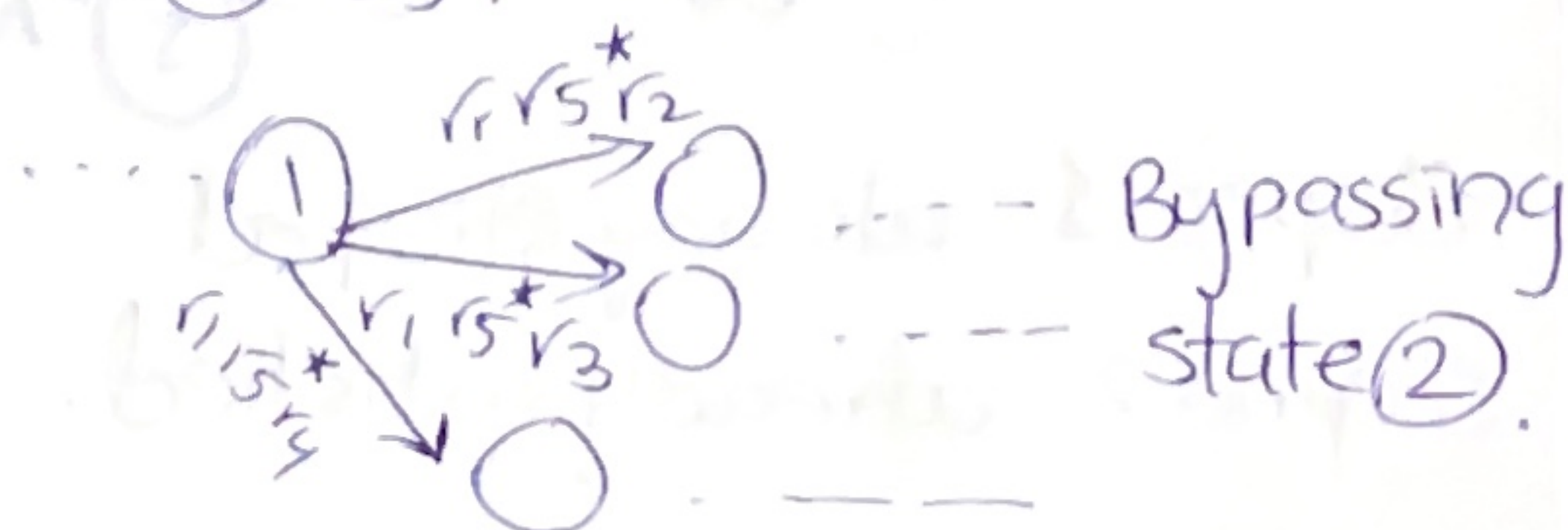
the change will occur as.



* Now consider if state (1) is connected to state (2) and state (2) is connected to more than one state i.e (3), (4), (5) then;



Eliminate edge from state (1) and (2) as:

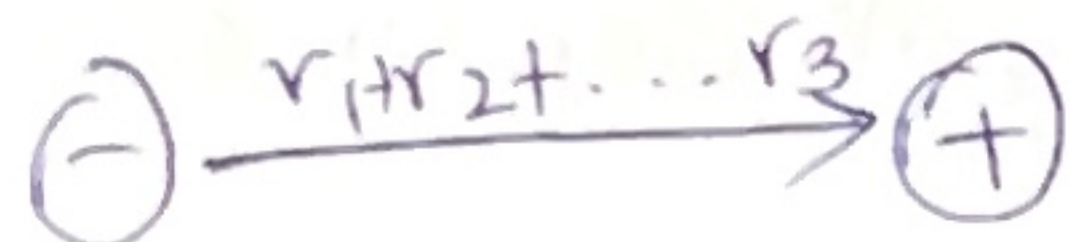


Bypassing state (2).

Note: Every state that leads to state ② can be made to Bypass state ②.

* The whole process can be repeated again and again until we have eliminated all the states from TG except for the unique start and final states are formed.

i.e

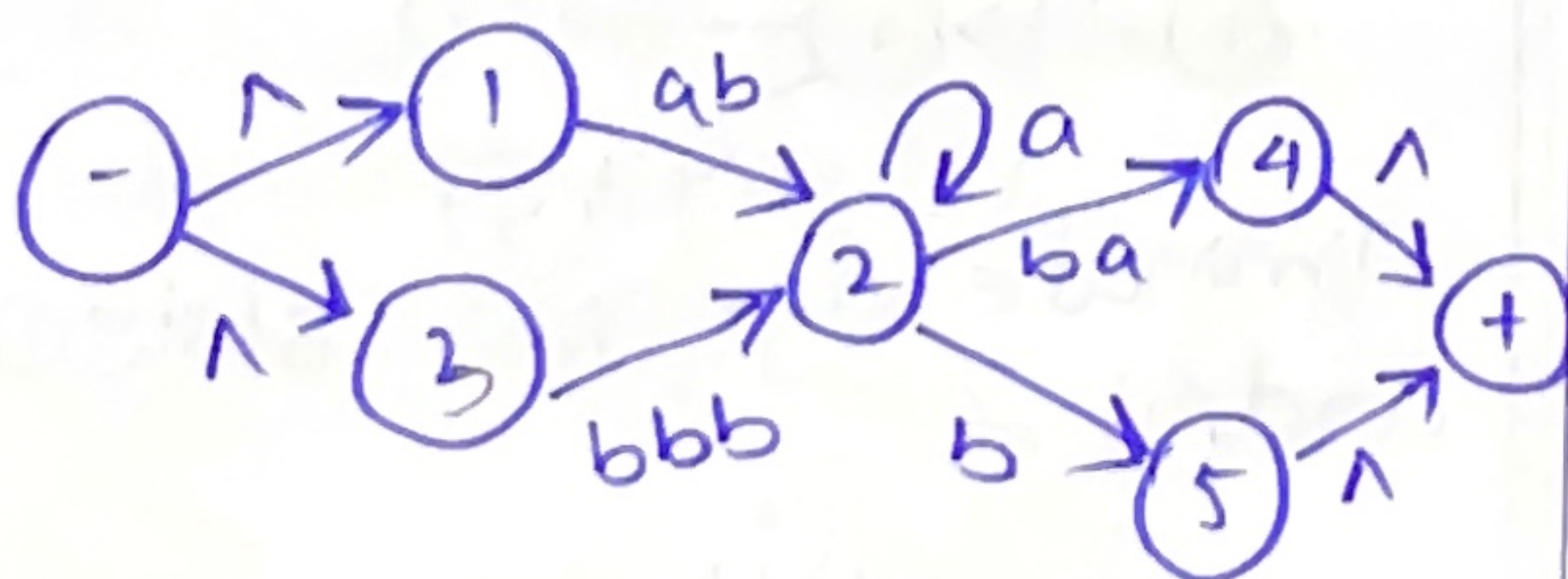


Resultant:- Would be a

RE that defines the same language as described/defined by the TG originally.

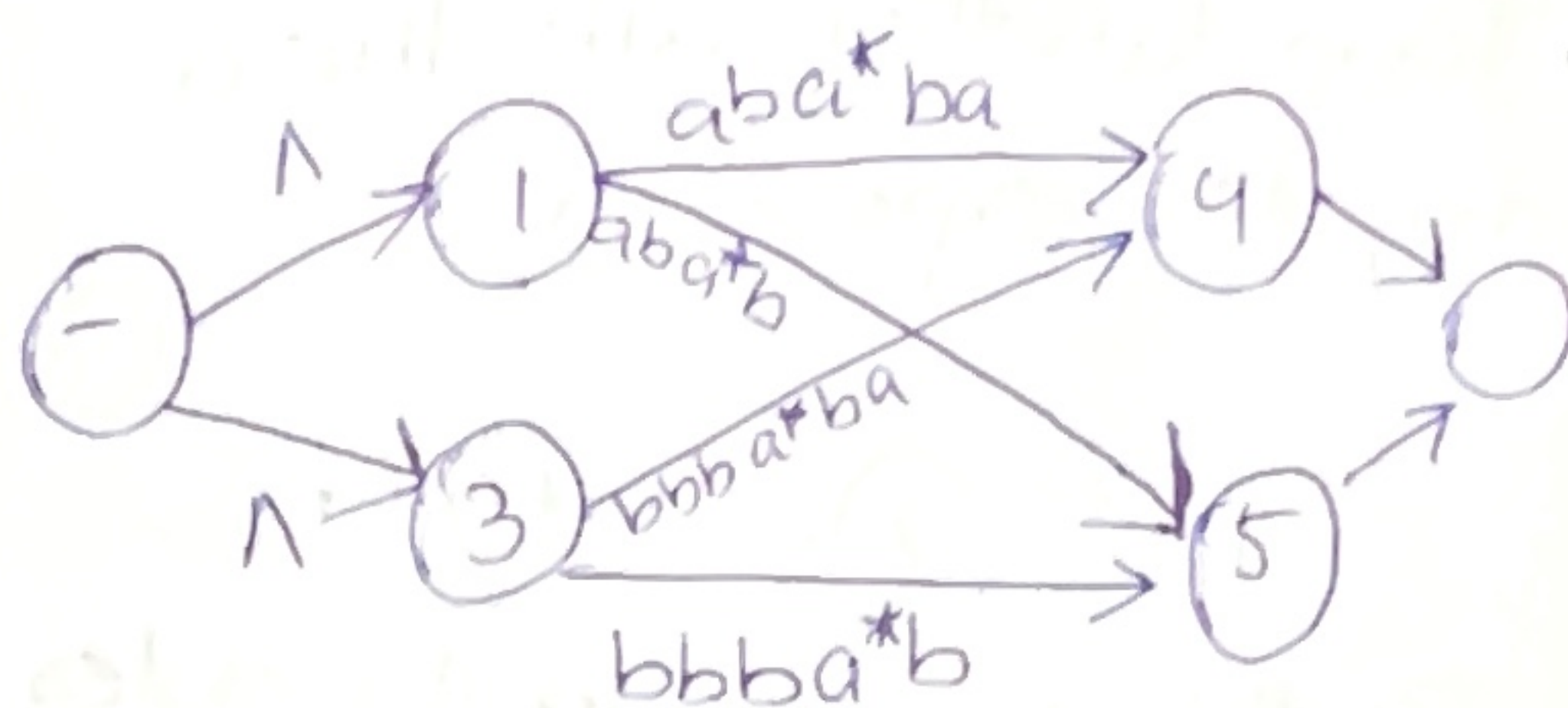
EXAMPLE

Convert TG into RE



Step no. 1 already satisfied
Step no. 2 already satisfied.

Bypass state 2.



state ① to ④ labelled with

aba^*ba .

state ④ to ⑤ labelled with

aba^*b .

state ③ to ④ labelled with

$bbba^*ba$

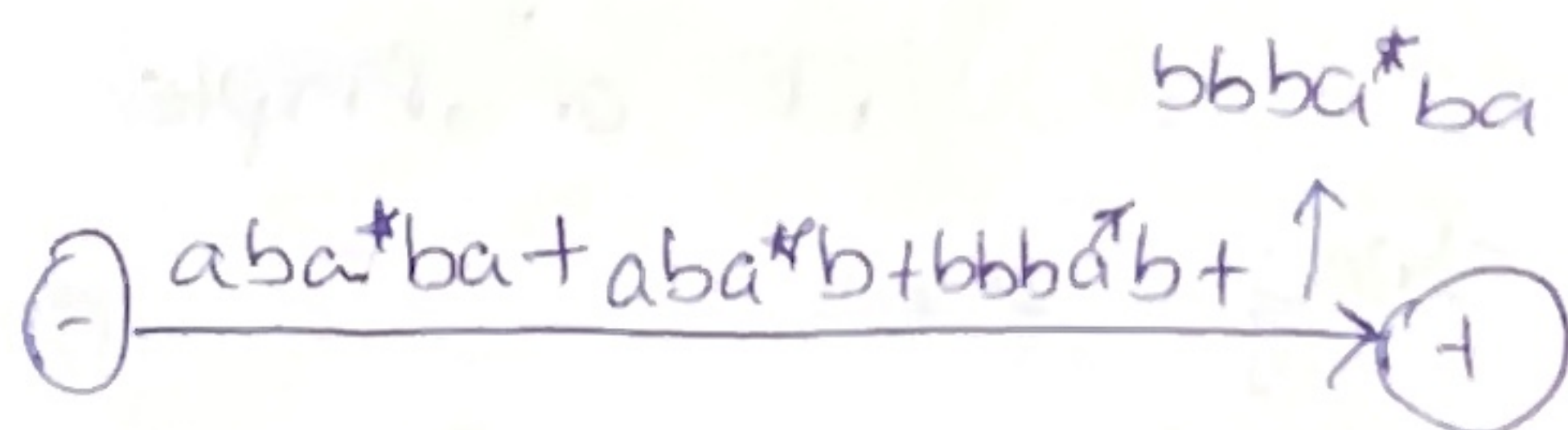
state ③ to ⑤ labelled with.

$bbba^*b$.

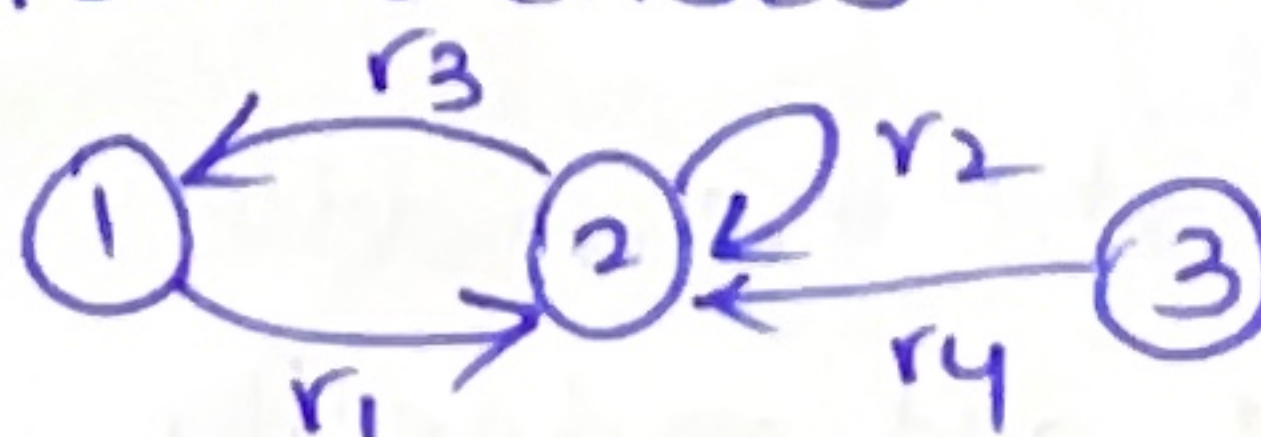
Note:

List down all edges leaving or coming into the state to eliminate.

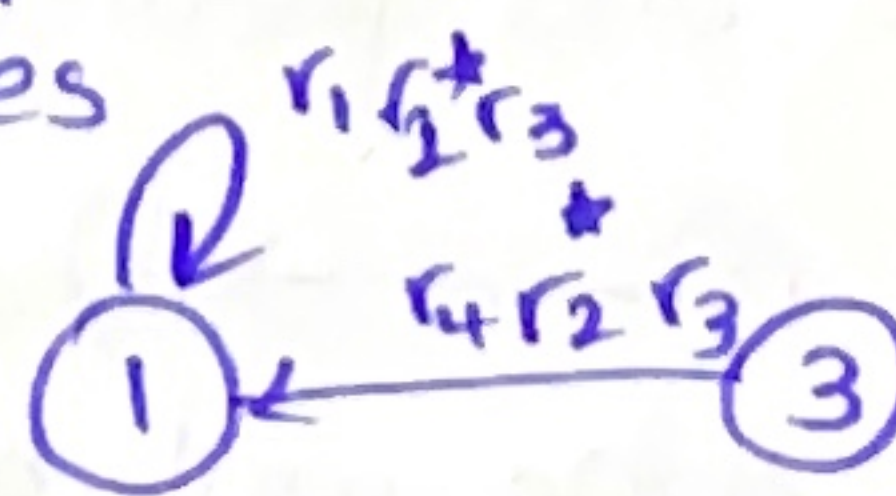
Result:-



SPECIAL CASES.



Becomes

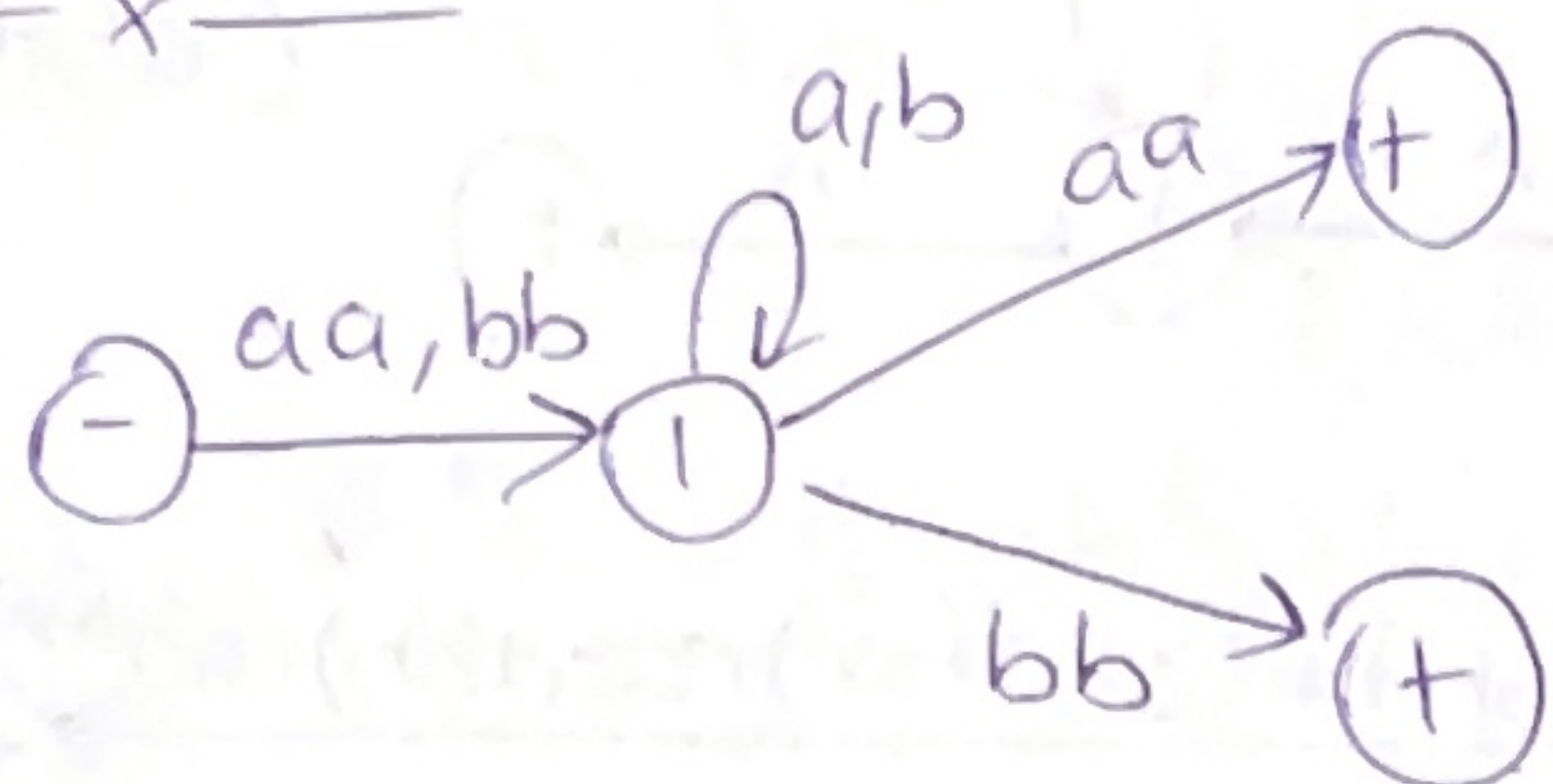


P.T.O
continued!

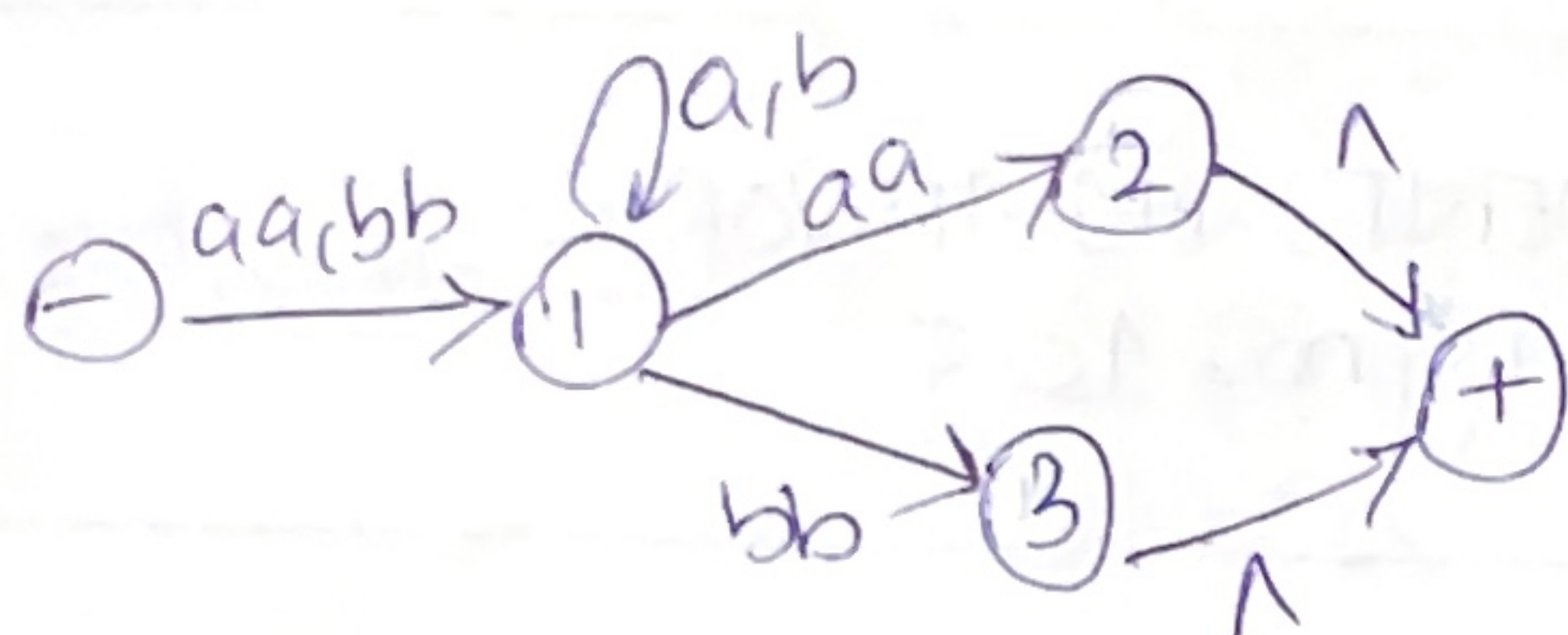
we could also do

$r_1 r_2^* r_3 \quad r_1 r_2^* r_3$
 $\swarrow$
 same loop over ①.

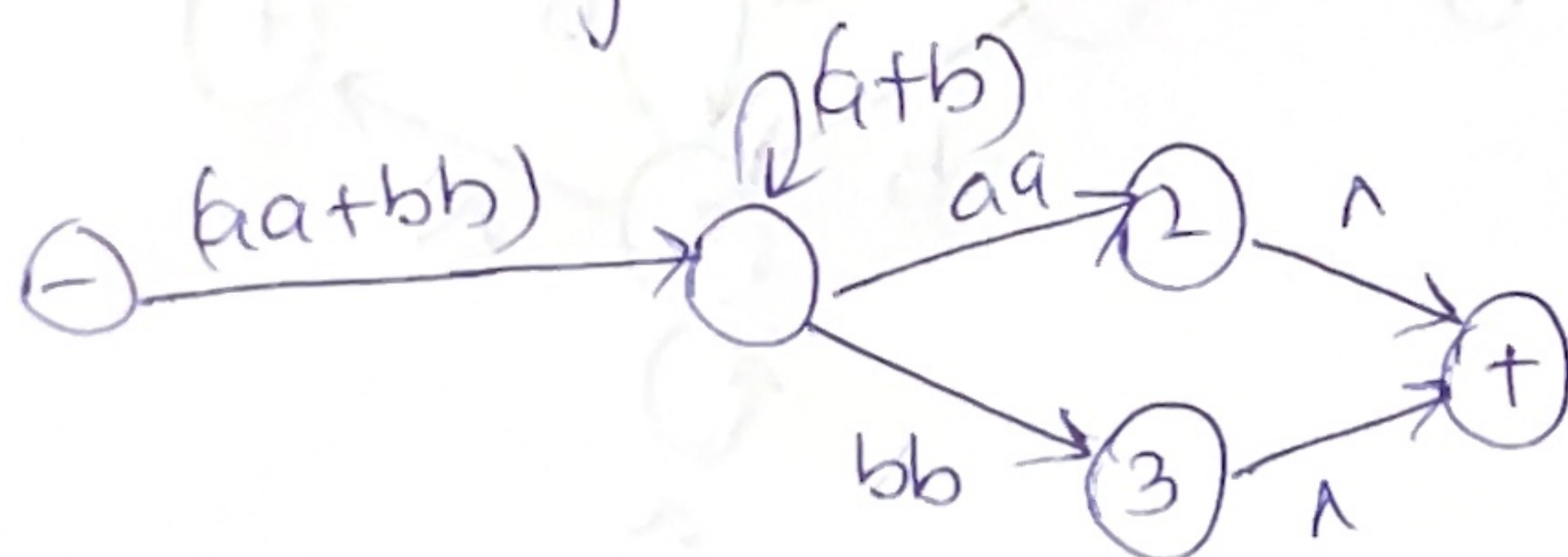
EXAMPLE:-



Step no. 1 start/initial & final states should be unique.

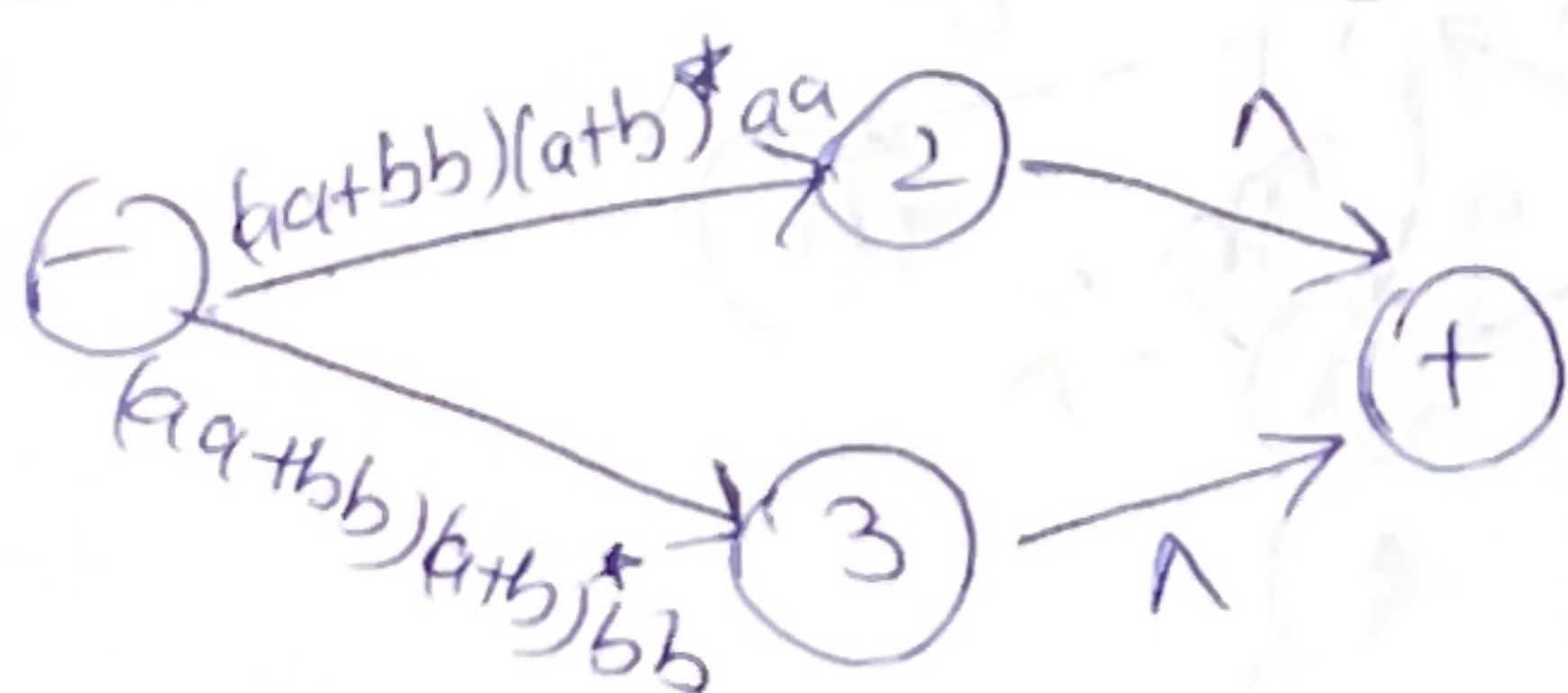


Step no. 2 Eliminate double edges.

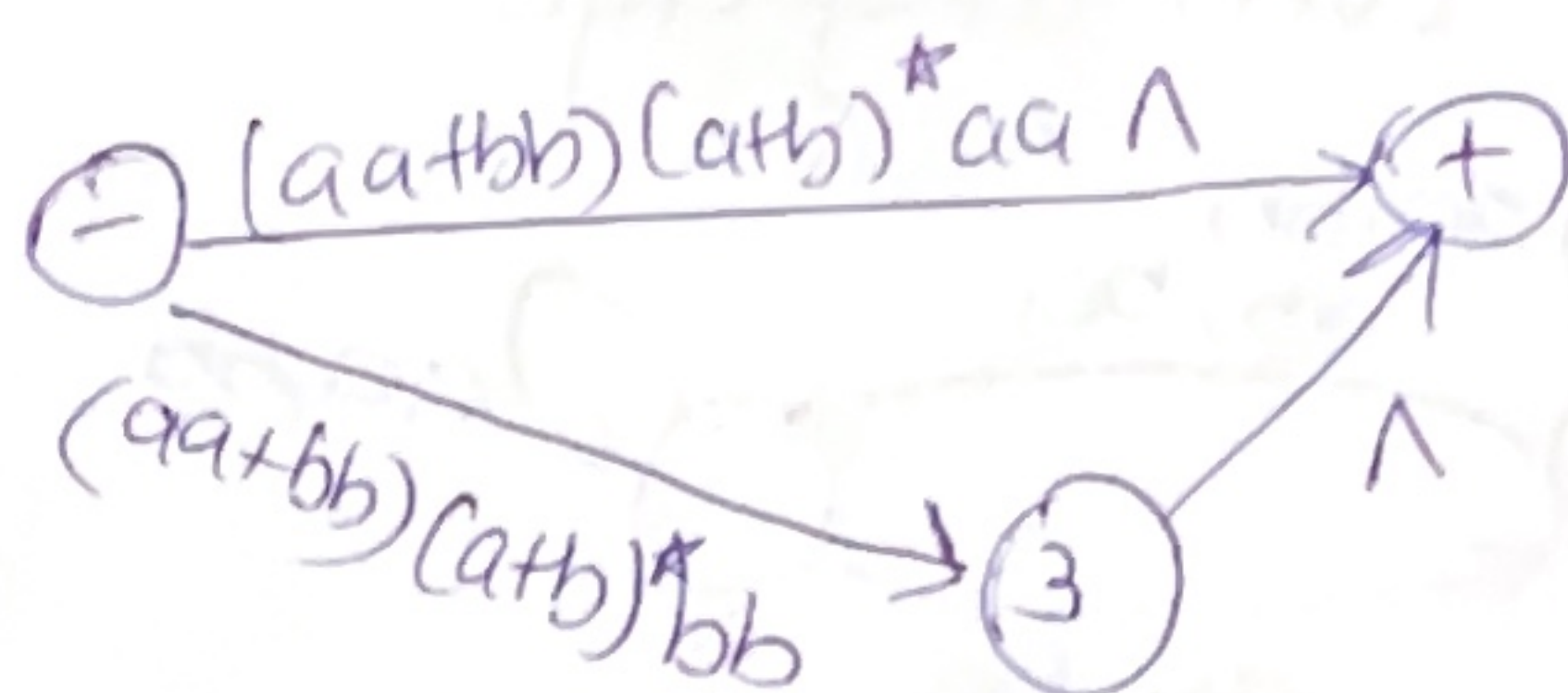


Step no. 3 By pass states.

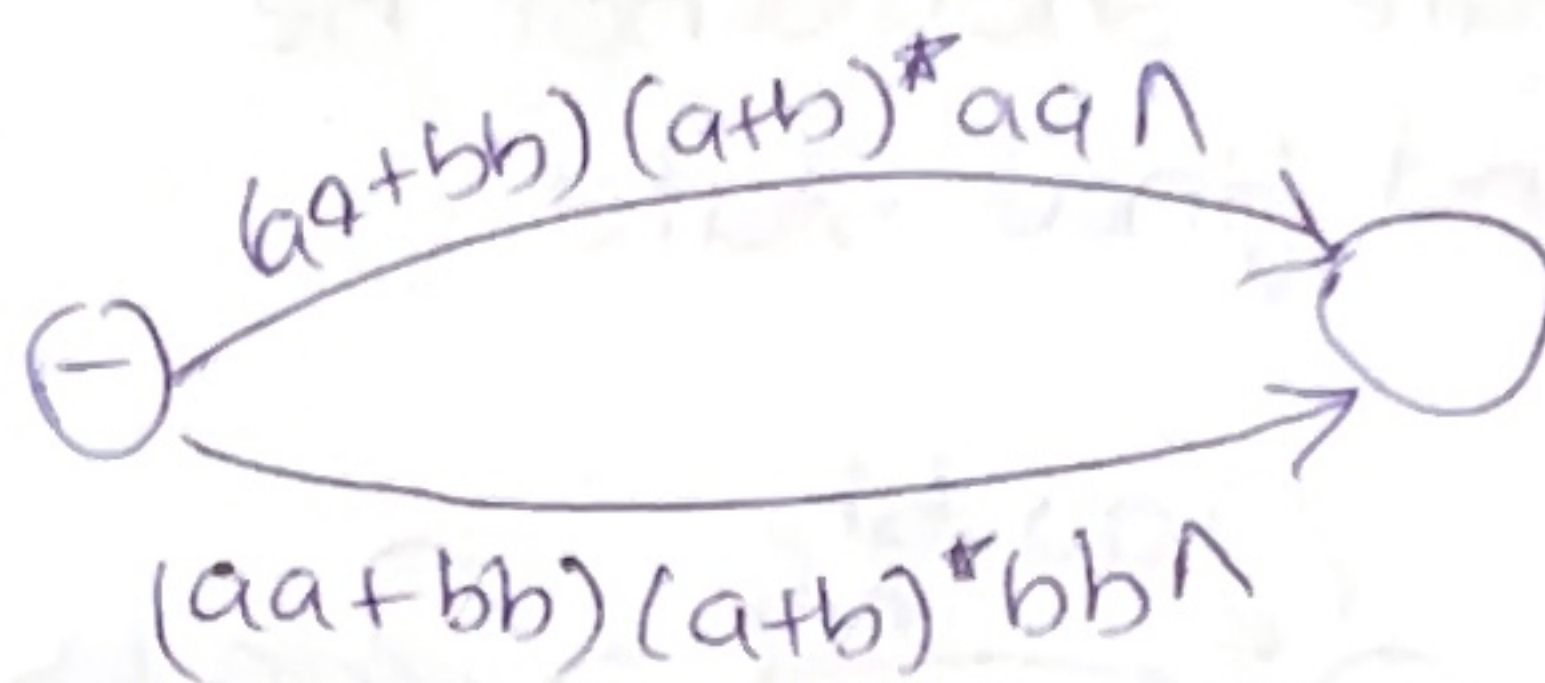
① Eliminate state ①.



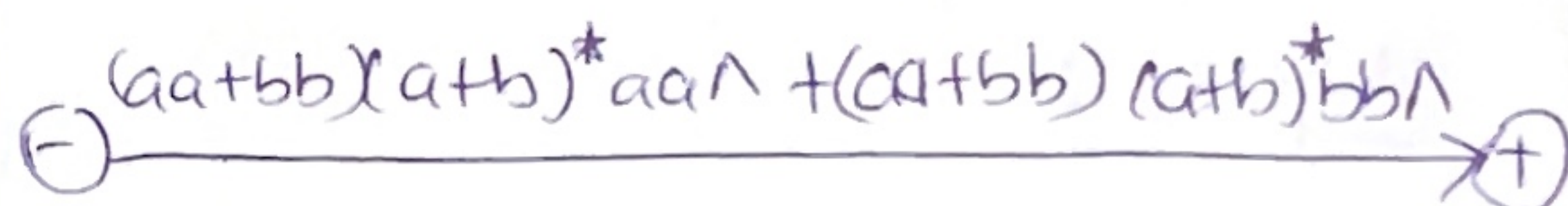
② Eliminate state ②



③ Eliminate state ③



④ Two edges going to same state.



$$\therefore RE \Rightarrow (aa+bb)(a+b)^*aa + (aa+bb)(a+b)^*bb$$

Actually we can also write it as

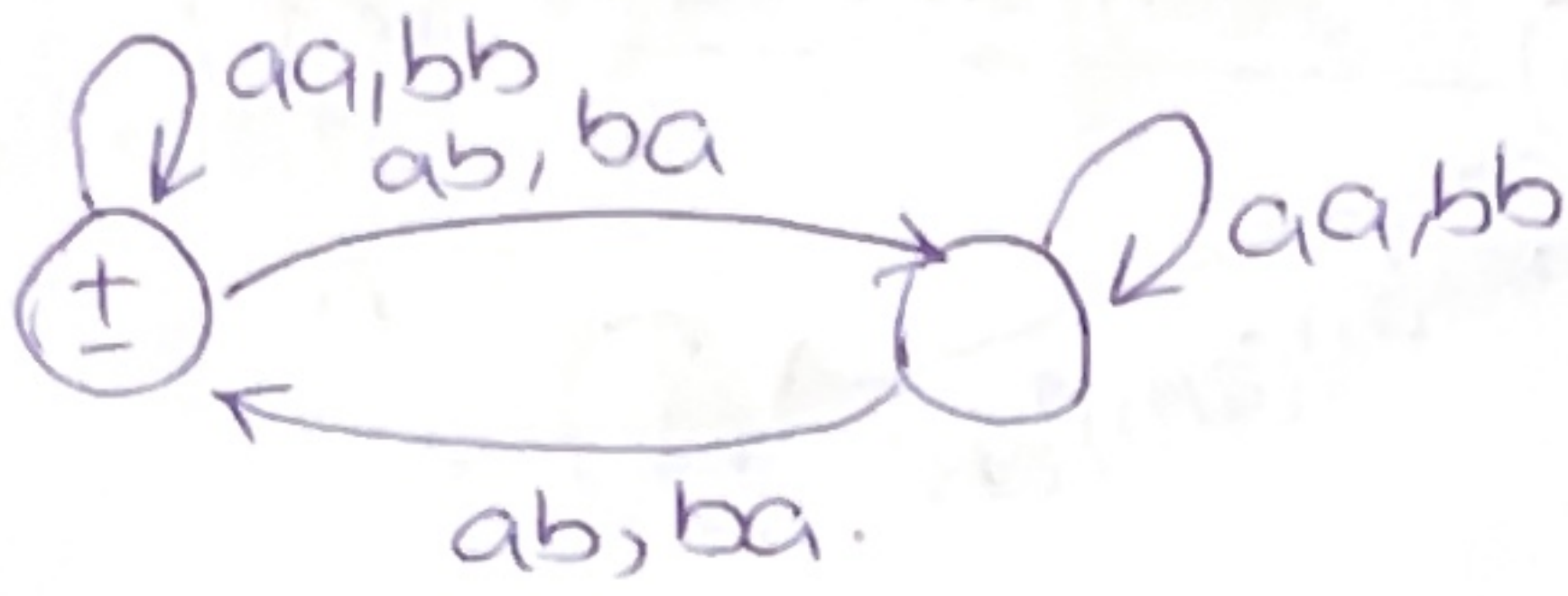
$$(aa+bb)(a+b)^*(aa+bb).$$

→ language of all the strings that begin and end with different letters?

~x~

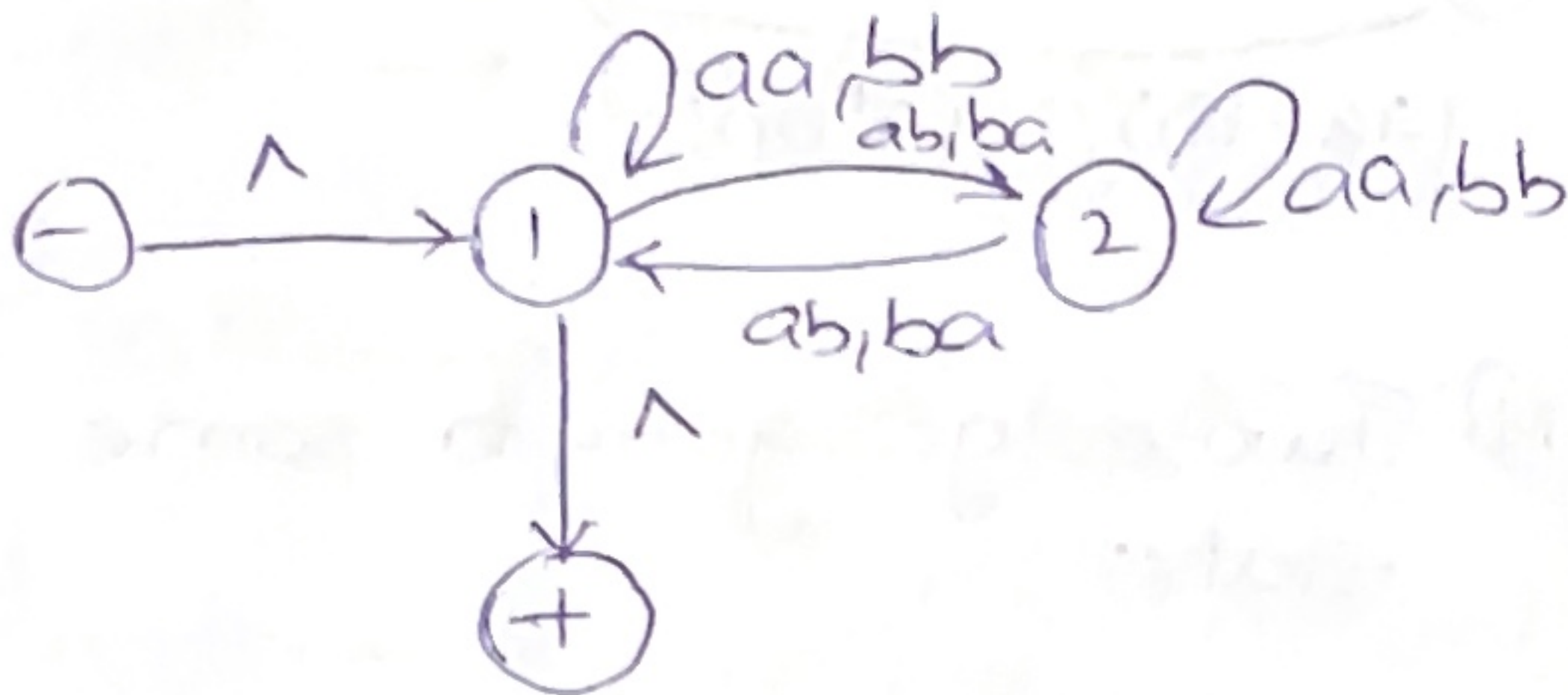
EXAMPLE

Even-Even language.



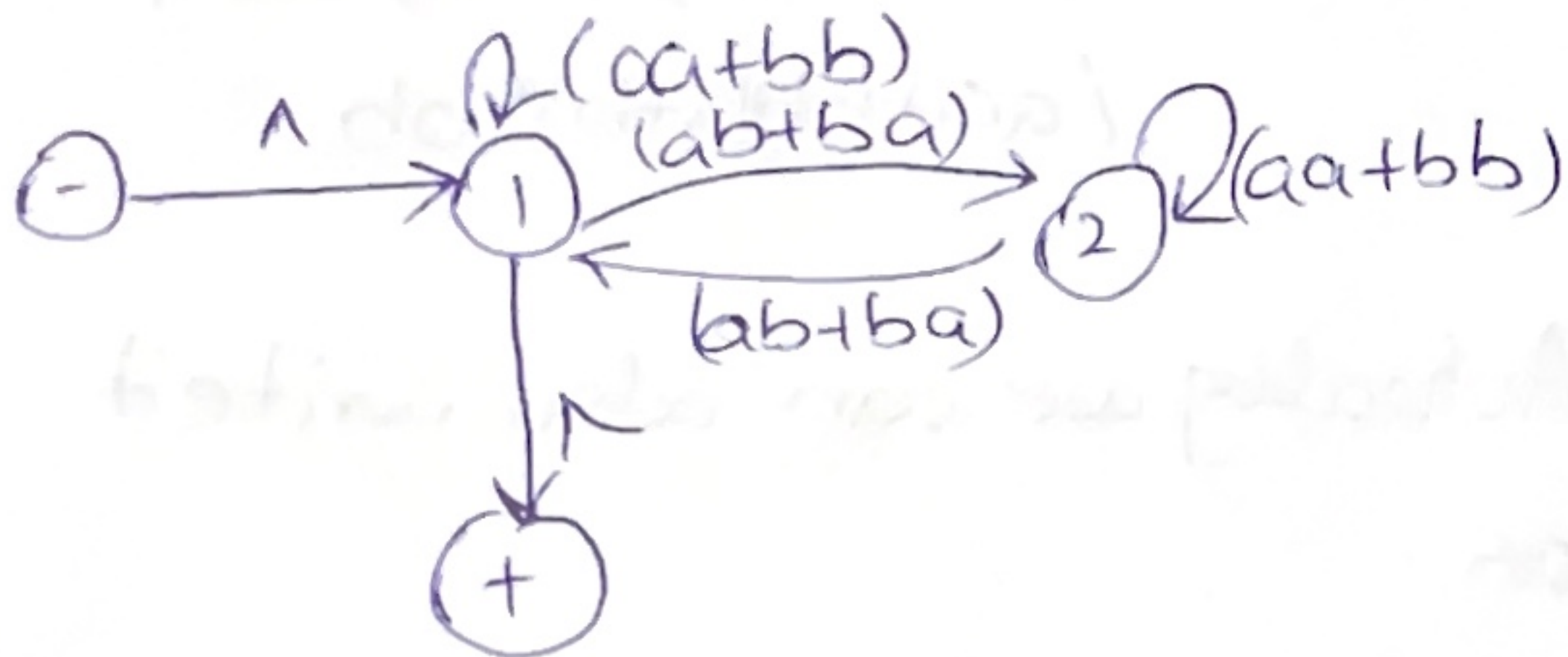
Step no. 1

Same state should not be initial and final state.



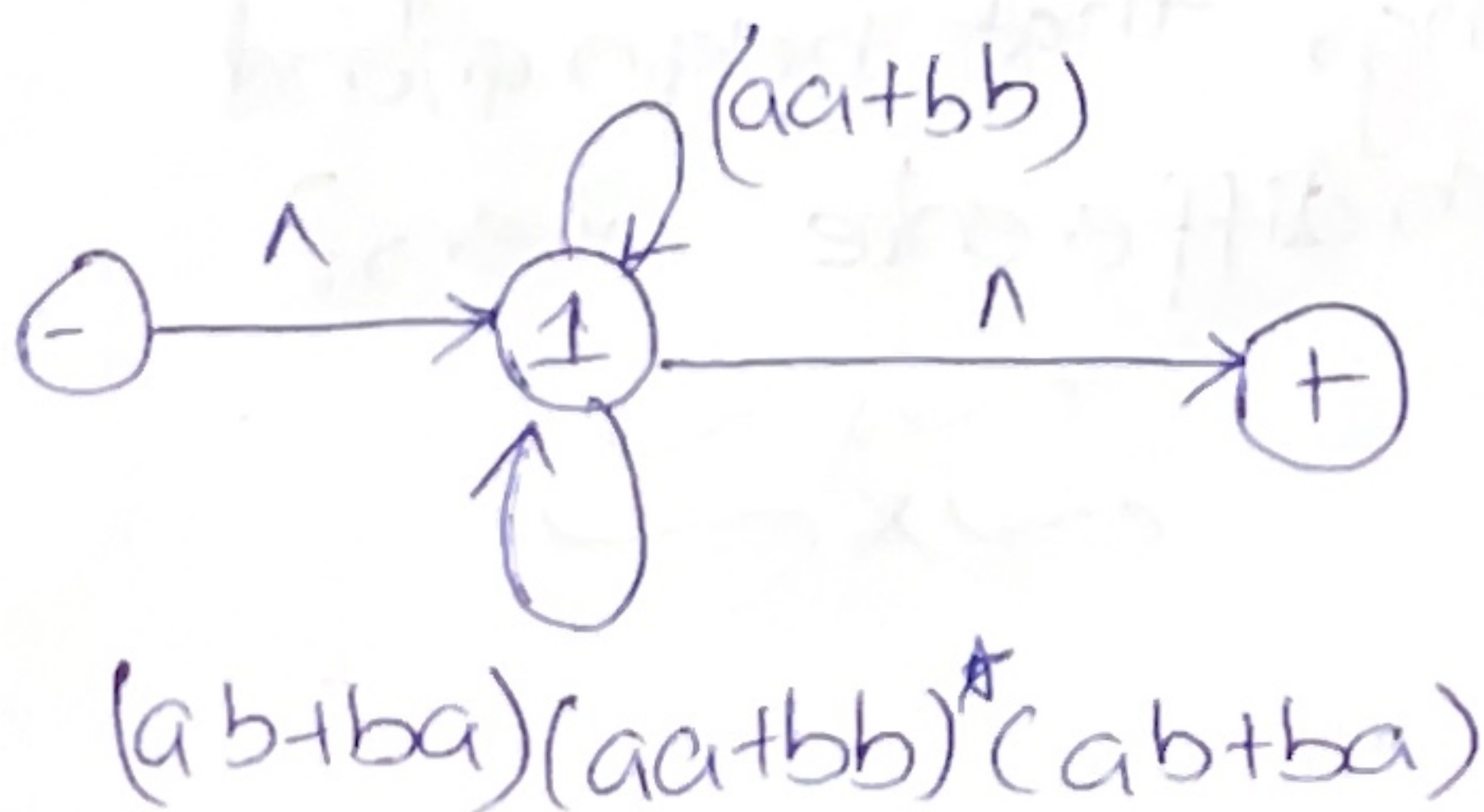
Step no. 2

Eliminate all double edges.

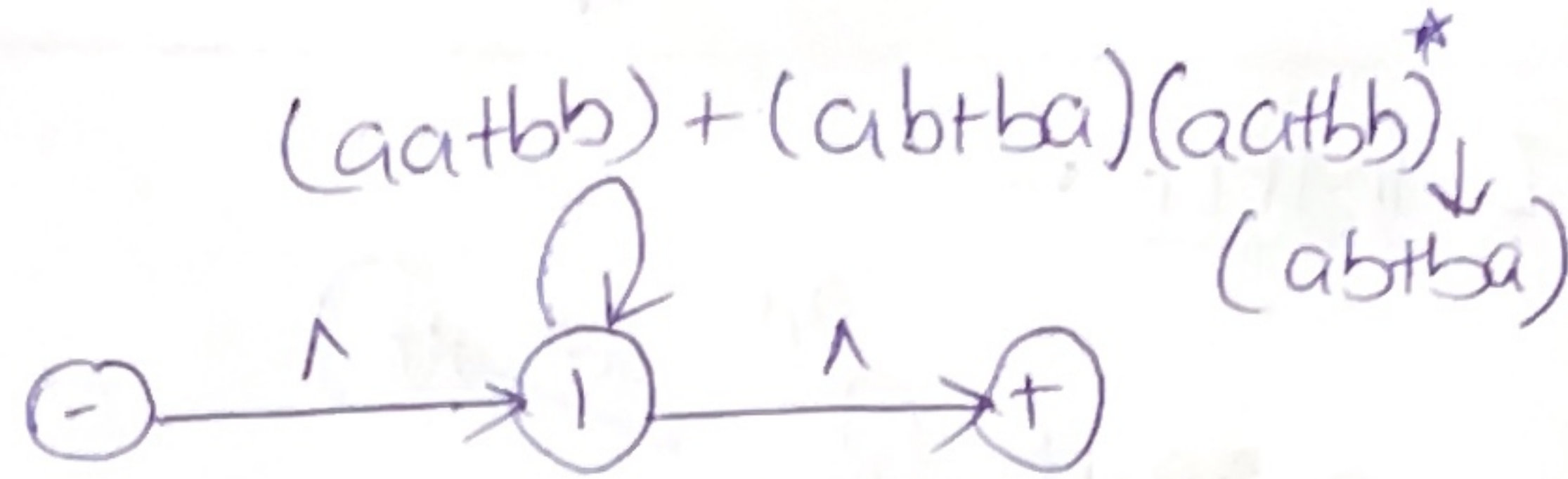


Step no. 3

By pass state no (2)



iii, Eliminate double edges on state (1)



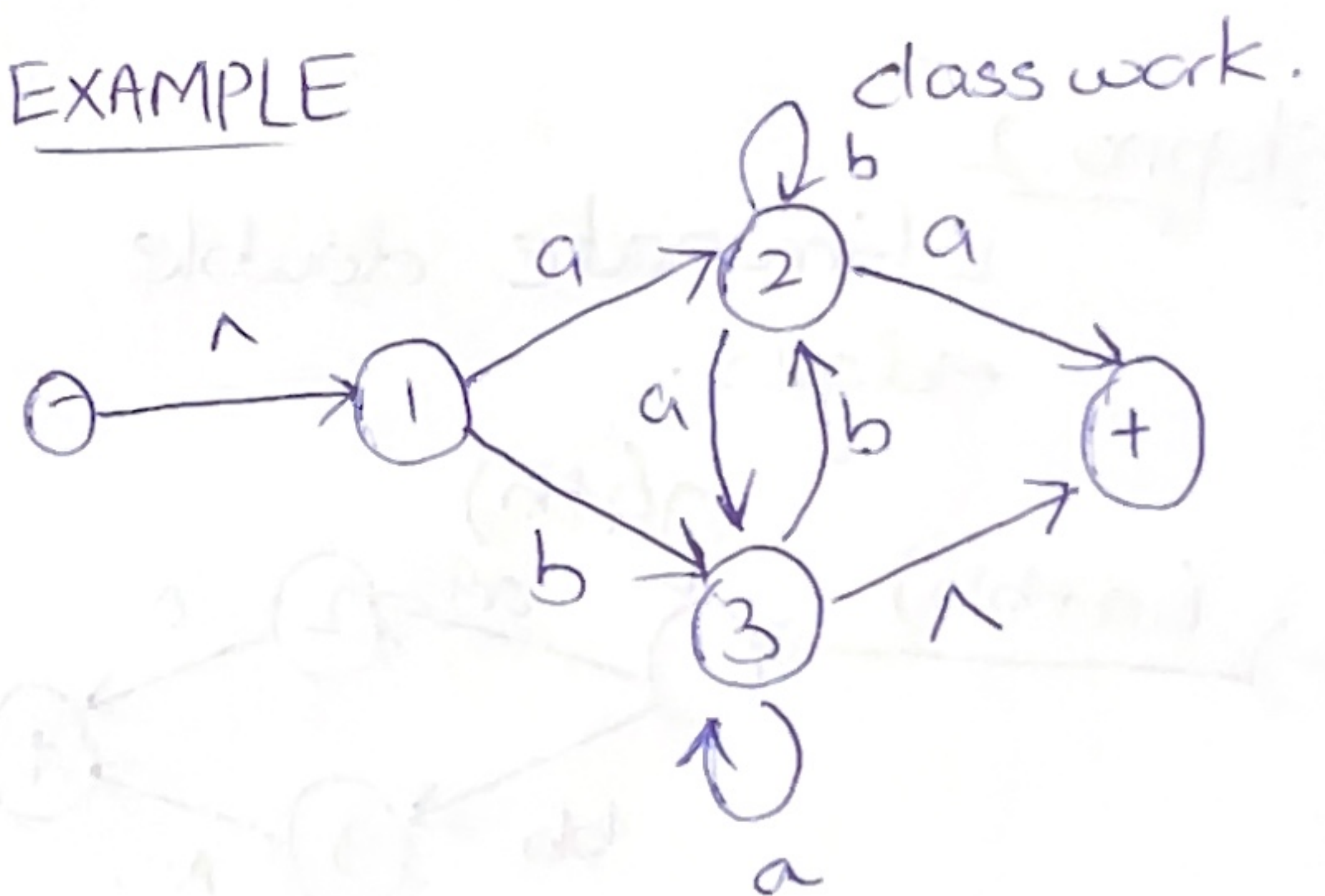
$$(aa+bb) + (ab+ba)(aa+bb)^*(ab+ba)$$

∴ The R.E is

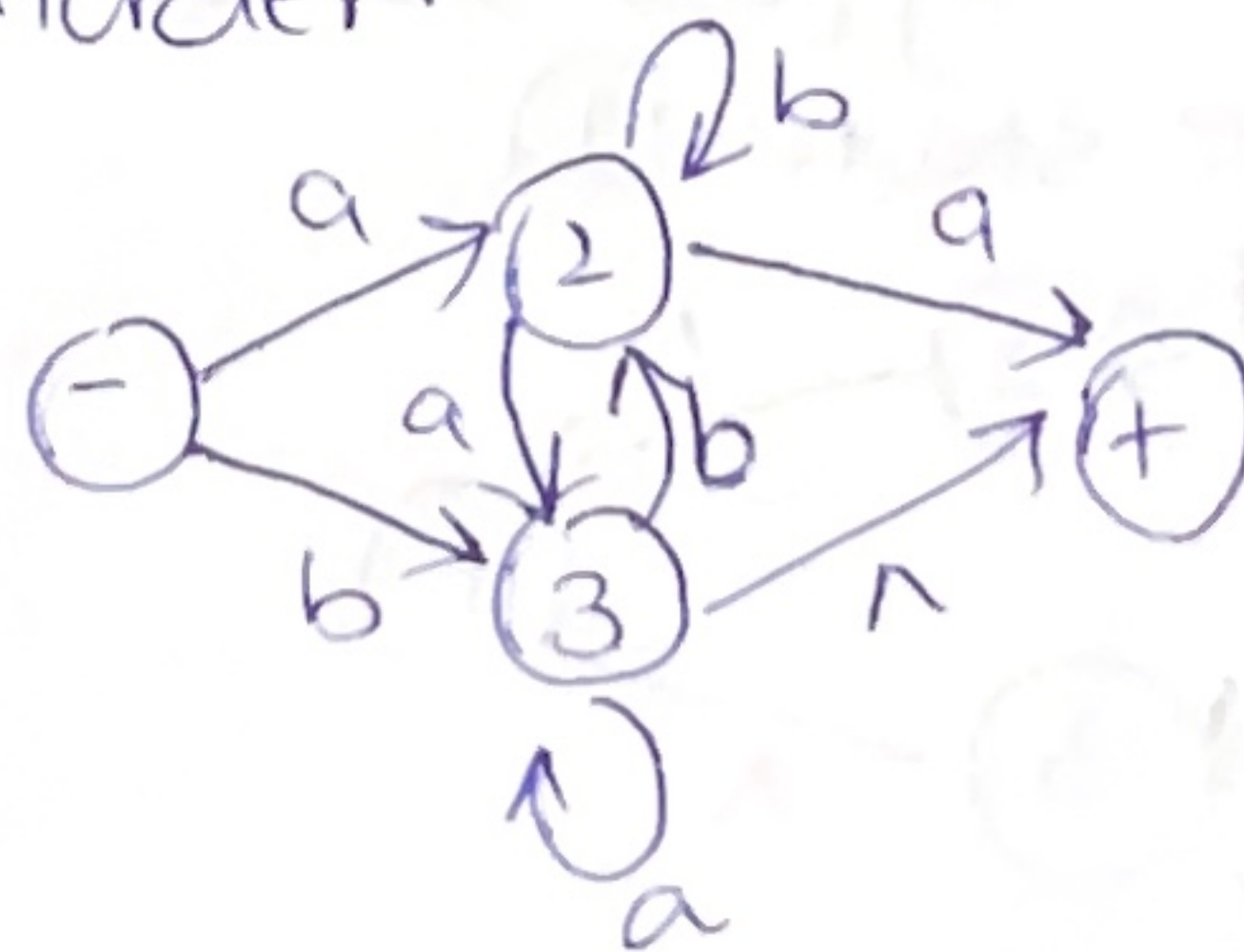
$$(aa+bb) + (ab+ba)(aa+bb)^*(ab+ba)$$

STUDENT HOMEWORK
pg no. 105

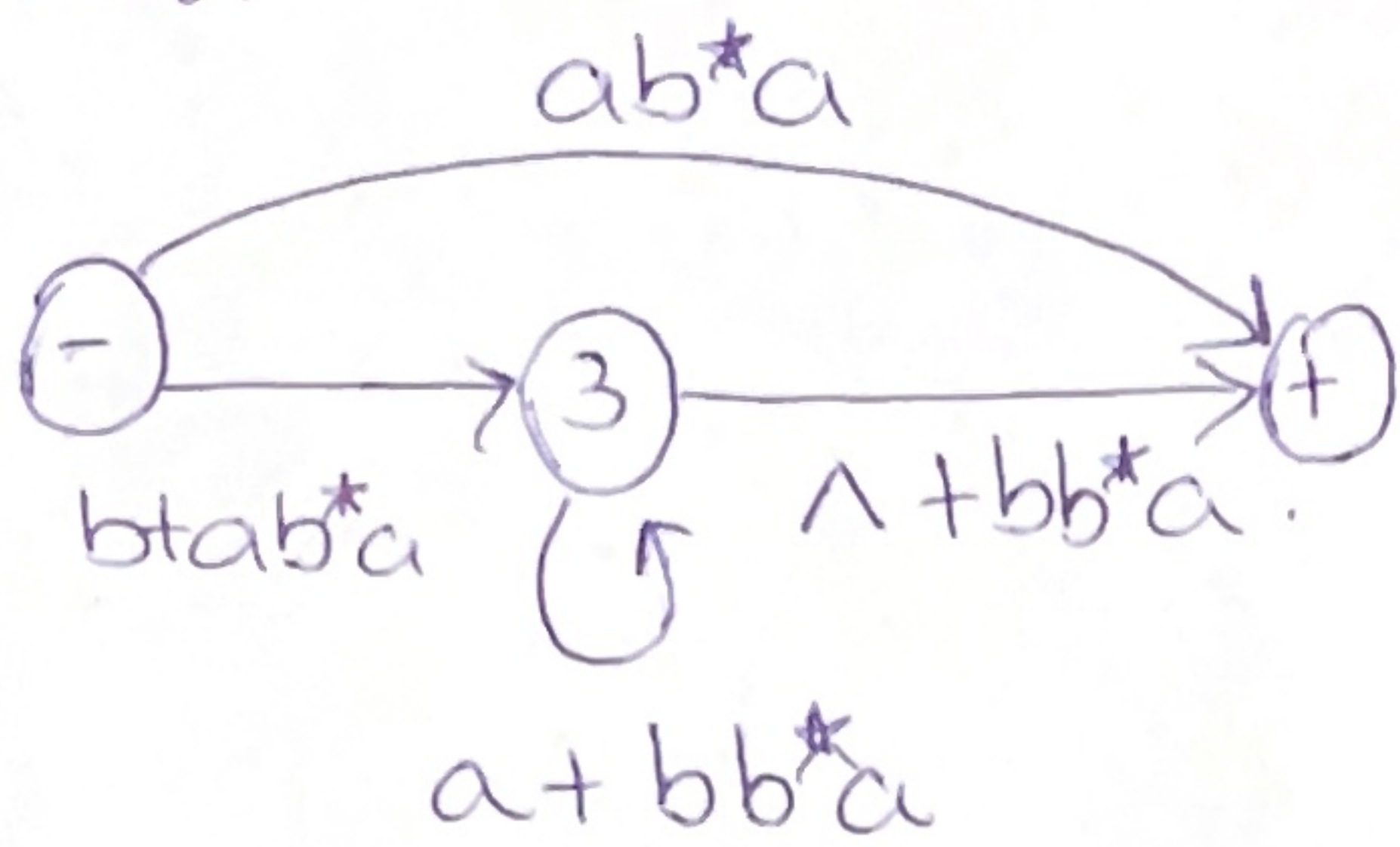
EXAMPLE



Eliminating state (1) (2) & (3) in order.



Next.



$$\ominus (ab^*a) + (btab^*a)(a+bb^*a)^+ \rightarrow \oplus$$

1-path $\rightarrow \ominus, \textcircled{2}, \oplus$ $\xrightarrow{ab^*a}$

2nd path $\rightarrow \ominus, \textcircled{3}, \textcircled{2}, \textcircled{3}, \oplus$

3rd path $\rightarrow \ominus, \textcircled{2}, \textcircled{3}, \oplus$ $\xrightarrow{b(a^*+bb^*a)}$

4th path $\rightarrow$